
Prueba de entrada (5 min.)



<https://forms.gle/1KkWTX7ye4mCQ1ZR8>

Aproximaciones ágiles

INFO248

Dra. Valeria Henríquez

~1990-2000: Reacción - Manifiesto Ágil

Manifiesto por el Desarrollo Ágil de Software

Estamos descubriendo formas mejores de desarrollar software tanto por nuestra propia experiencia como ayudando a terceros. A través de este trabajo hemos aprendido a valorar:

Individuos e interacciones sobre procesos y herramientas
Software funcionando sobre documentación extensiva
Colaboración con el cliente sobre negociación contractual
Respuesta ante el cambio sobre seguir un plan
Esto es, aunque valoramos los elementos de la derecha, valoramos más los de la izquierda.

- Formulado en 2001.
- Respuesta a los problemas de los procesos tradicionales.

¡Iniciado por la industria, no por la Academia!

Kent Beck	James Grenning	Robert C. Martin
Mike Beedle	Jim Highsmith	Steve Mellor
Arie van Bennekum	Andrew Hunt	Ken Schwaber
Alistair Cockburn	Ron Jeffries	Jeff Sutherland
Ward Cunningham	Jon Kern	Dave Thomas
Martin Fowler	Brian Marick	

<https://agilemanifesto.org/iso/es/manifesto.html>

~1990-2000: Reacción - Manifiesto Ágil

Las **aproximaciones** de desarrollo ágiles proporcionan
FLEXIBILIDAD EN EL DESARROLLO

Desarrollo orientado al **USUARIO** (y al **VALOR** que se le proporciona)

Prima el **PRODUCTO** sobre la documentación

Desarrollo centrado en las
PERSONAS

Desarrollo basado en un
Ciclo de Vida
ITERATIVO/INCREMENTAL



Programación Extrema (eXtreme Programming - XP)

XP - Orígenes

XP fue creado por Kent Beck a finales de la década de 1990. Surgió como respuesta a las limitaciones de las metodologías tradicionales de desarrollo de software.

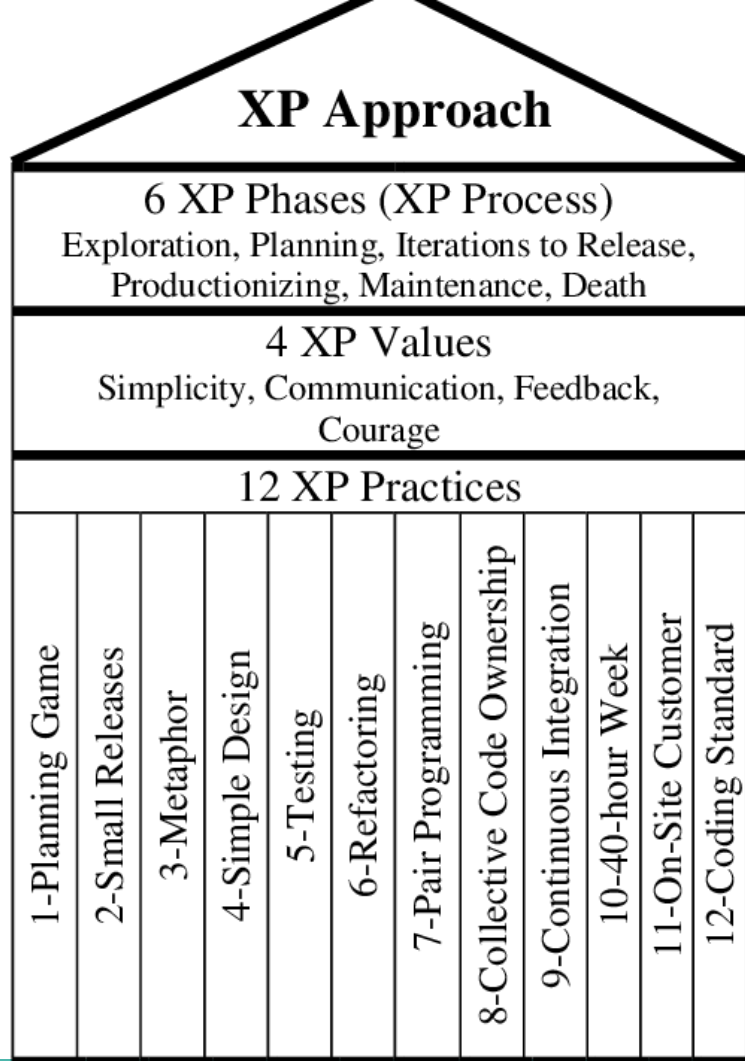
XP se basa en los principios de simplicidad, comunicación, retroalimentación continua y valentía.

XP fue uno de los primeros métodos ágiles; de hecho, XP fue el método ágil dominante a finales de los 90 y principios de los 2000 antes de que Scrum se volviera dominante.

Fundamentos de XP

Esta metodología se basa en valores y prácticas.

"No soy un gran programador; Solo soy un buen programador con buenos hábitos". - Kent Beck



Principios de XP - Planning Game

Proceso de planificación: el usuario tendrá que escribir sus necesidades, definiendo las actividades que realizará el sistema. Se creará un **documento llamado Historias del usuario (User Stories)**. Entre **20 y 80 historias** (todo dependiendo de la complejidad del problema) se consideran suficientes para formar el llamado **Plan de Liberación**, el cual define de forma específica los tiempos de entrega de la aplicación para recibir retroalimentación por parte del usuario. Por regla general, cada **una de las Historias del usuario suelen necesitar de una a tres semanas de desarrollo**.

Son muy importantes y tienen que ser una constante las reuniones periódicas durante esta fase de planificación. Estas pueden ser a diario, con todo el equipo de desarrollo para identificar problemas, proponer soluciones y señalar aquellos puntos a los que se les ha de dar más importancia por su dificultad o por su punto crítico.

Principios de XP - Small Releases

Entregas pequeñas: colocar un sistema sencillo en producción rápidamente que se actualiza de forma rápida y constante permitiendo que el verdadero valor de negocio del producto sea evaluado en un ambiente real. Estas entregas no pueden pasar las **2 o 3 semanas como máximo**.

Principios de XP - Metaphor

Metáfora: desarrollada por los programadores al inicio del proyecto, define una historia de cómo funciona el sistema completo. XP estimula historias, que son breves descripciones de un trabajo de un sistema en lugar de los tradicionales diagramas y modelos UML (Unified Modeling Language). La metáfora expresa la visión evolutiva del proyecto que define el alcance y propósito del sistema.

Las tarjetas CRC (Clase, Responsabilidad y Colaboración) también ayudarán al equipo a definir actividades durante el diseño del sistema. Cada tarjeta representa una clase en la programación orientada a objetos y define sus responsabilidades (lo que ha de hacer) y las colaboraciones con las otras clases (cómo se comunica con ellas).

Debemos tener la capacidad de explicar el diseño del sistema a personas nuevas a través de metáforas en vez de pedirles que lean una gran cantidad de documentos.

Principios de XP - Simple Design

Diseño simple: se basa en la filosofía de que el mayor valor de negocio es entregado por el sistema más sencillo que cumpla los requerimientos. Simple Design se enfoca en **proporcionar un sistema que cubra las necesidades inmediatas del cliente, ni más ni menos**. Este proceso permite eliminar redundancias y rejuvenecer los diseños obsoletos de forma sencilla

Principios de XP - Testing

El principio de pruebas: se tiene que establecer un período de pruebas de aceptación del programa (llamado también período de caja negra) donde se definirán las entradas al sistema y los resultados esperados de estas entradas. Es muy recomendable automatizar estas pruebas para poder hacer varias simulaciones del sistema en funcionamiento

Desarrollo guiado por pruebas (*TDD, Test Driven Development*)

Principios de XP - Refactoring

Refactorización: permite a los equipos de programadores XP mejorar el diseño del sistema a través de todo el proceso de desarrollo. Los programadores evalúan continuamente el diseño y recodifican lo necesario. La finalidad es mantener un sistema enfocado a proveer el valor de negocio mediante la minimización del código duplicado y/o ineficiente.

Mejorar el diseño del código existente sin cambiar su comportamiento.

Principios de XP - Pair programming

Programación en parejas: uno de los principios más radicales y en el que la mayoría de gerentes de desarrollo pone sus dudas. Requiere que todos los programadores XP escriban su código en parejas, compartiendo una sola máquina.

Dos programadores trabajando en pareja en una sola máquina, resolviendo el mismo problema.

Principios de XP - Collective Code Ownership

Propiedad colectiva del código: **un código con propiedad compartida**. Nadie es el propietario de nada, todos son el propietario de todo. Este método difiere en mucho a los métodos tradicionales en los que un simple programador posee un conjunto de código. Los defensores de XP argumentan que mientras haya más gente trabajando en una pieza, menos errores aparecerán.

Ningún miembro del equipo posee una parte específica del código fuente, el código es de todos.

Principios de XP - Continuous Integration

Integración continua: permite al equipo hacer un rápido progreso implementando las nuevas características del software. En lugar de crear builds (o versiones) estables de acuerdo a un cronograma establecido, los equipos de programadores XP pueden reunir su código y reconstruir el sistema varias veces al día. Esto reduce los problemas de integración comunes en proyectos largos y estilo cascada.

Ningún código permanece sin integrar durante más de un par de horas. -- Kent Beck

Debería ser capaz de caminar con una computadora portátil cargada con solo un sistema operativo, y al usar el repositorio, obtener todo lo que necesito para compilar y ejecutar el producto. – Martin Fowler

Principios de XP - 40 hour work week

La semana de 40 horas: la programación extrema sostiene que los programadores cansados escriben código de menor calidad. Minimizar las horas extras y mantener los programadores frescos, generará código de mayor calidad.

Está bien trabajar tiempos extra cuando es necesario, pero no se ha de hacer durante dos semanas seguidas. -- *Kent Beck*

Principios de XP - On Site Client

El cliente en el sitio: se le dará poder para determinar los requerimientos, definir la funcionalidad, señalar las prioridades y responder las preguntas de los programadores. Esta fuerte interacción cara a cara con el programador disminuye el tiempo de comunicación y la cantidad de documentación, junto con los altos costes de su creación y mantenimiento.

El representante del cliente estará con el equipo de trabajo durante toda la realización del proyecto.

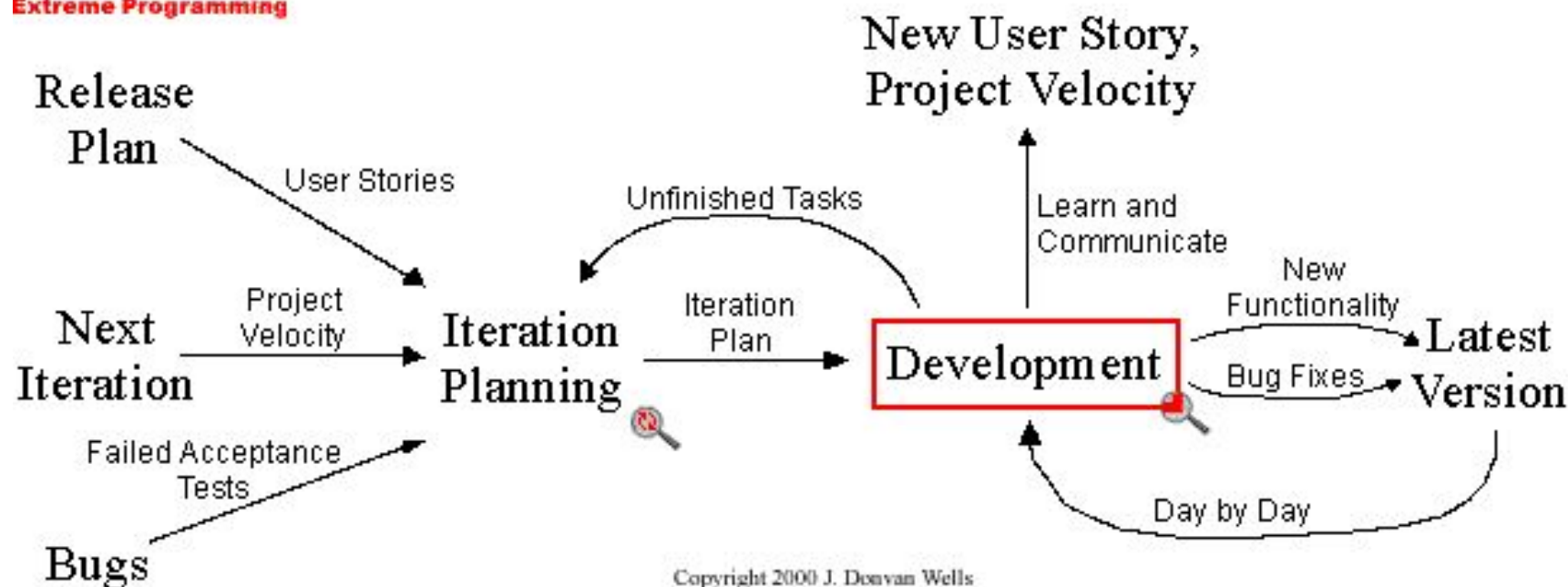
Principios de XP - Coding Standard

Estándar de codificación: define la gestión del código compartido, así como las reglas para escribir y documentar el código y la comunicación entre diferentes piezas de código desarrolladas por diferentes equipos. Los programadores las han de seguir de tal manera que el código en el sistema se vea como si hubiera estado escrito por una sola persona.

Su propósito es producir software que tenga un estilo consistente, independientemente del autor, lo que da como resultado un software que es más fácil de entender y mantener.



Iteration



¿Cuándo deberías implementar XP?

Suele ser implementada solamente por los equipos de ingeniería.

- **Para gestionar un equipo más pequeño.** Debido a su naturaleza altamente colaborativa, la programación extrema funciona mejor en equipos pequeños de menos de diez personas.
- **Si estás constantemente en contacto con tus clientes.** La programación extrema incorpora los requisitos de los clientes a lo largo del proceso de desarrollo y también se basa en ellos para las pruebas y aprobaciones.
- **Si trabajas con un equipo flexible que pueda aceptar el cambio (sin resentimientos).** Dada su propia naturaleza, la programación extrema a menudo requerirá que todo el equipo deseche todo su arduo trabajo. Algunas reglas también permiten que algunos miembros del equipo realicen cambios en cualquier momento, lo que supondría un problema si los demás compañeros del equipo se lo toman como algo personal.
- **Si dominas los aspectos técnicos de la codificación.** La programación extrema no es para principiantes ya que necesitas poder trabajar e implementar cambios rápidamente.

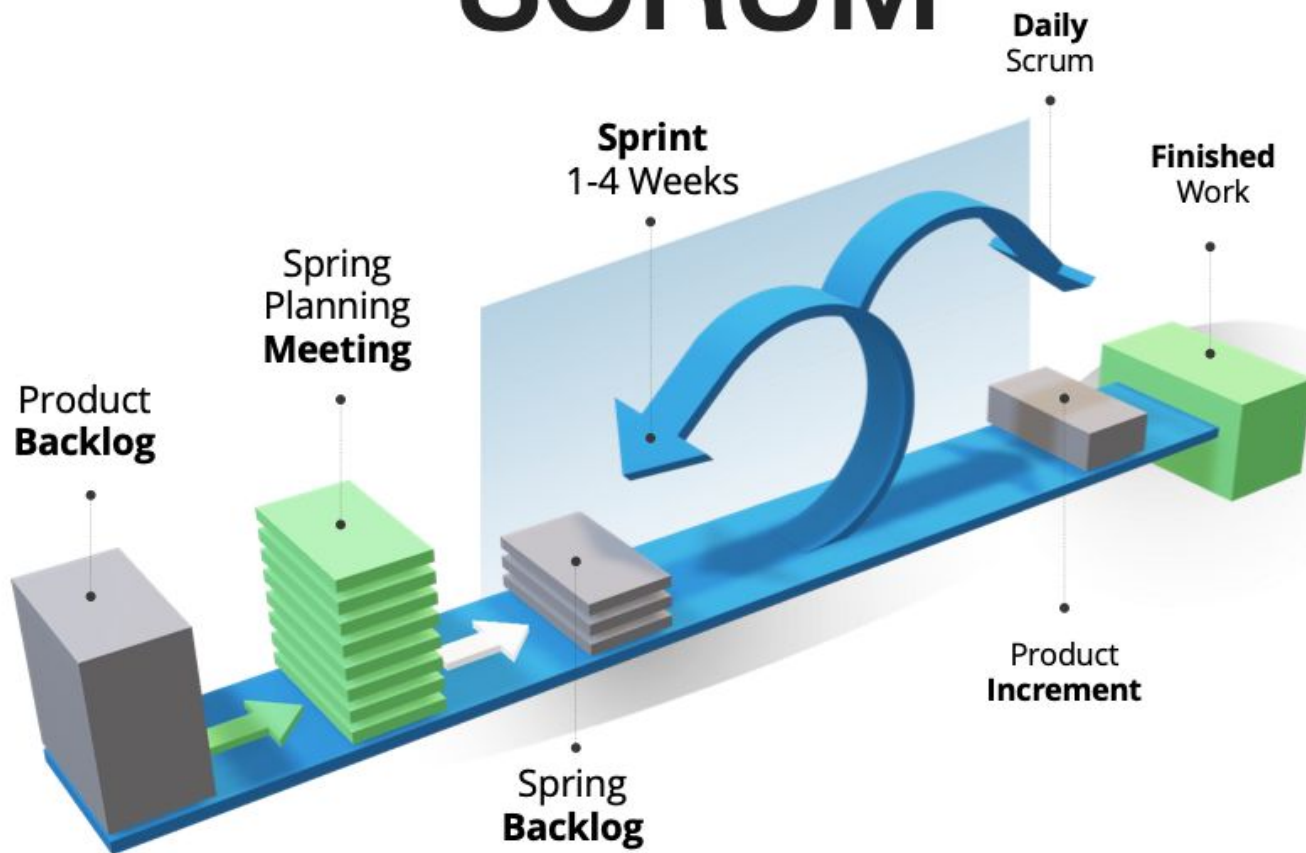
Scrum

Scrum

- Origen: 1986, Harvard Business Review (Takeuchi & Nonaka)
 - Nuevo enfoque para el desarrollo de productos comerciales
 - Objetivo: incrementar la **flexibilidad** y **velocidad** de desarrollo
 - Enfoque “holístico” o “scrum”

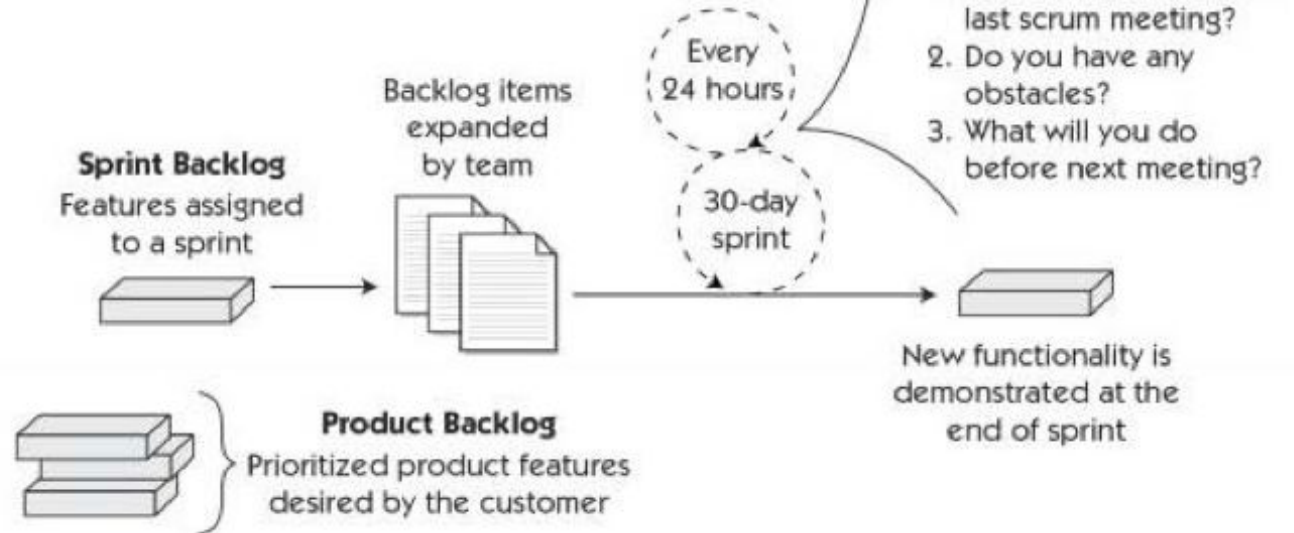


SCRUM



Scrum: Eventos (meetings)

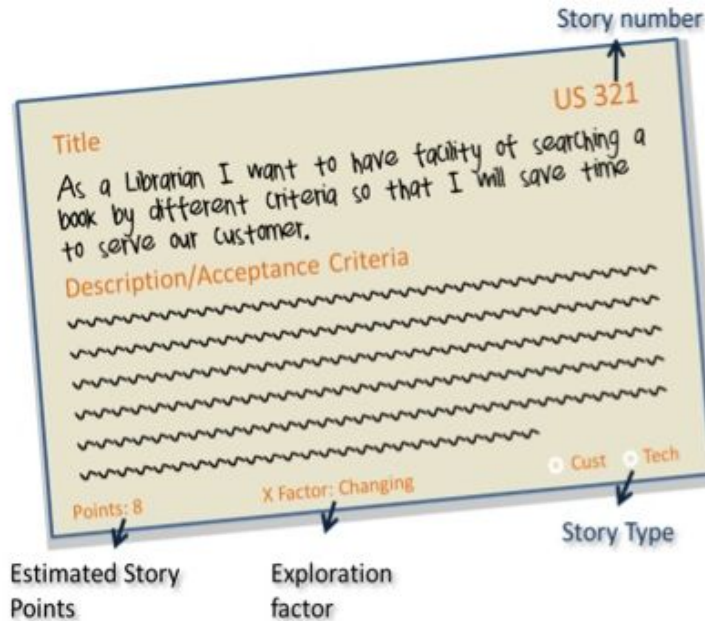
- Antes del Sprint: Sprint Planning
- Durante el sprint: Daily Meetings
- Después del sprint: Sprint Review
- Después del sprint: Sprint Retrospective



Scrum: artefactos

As a <User>, I want to <Have> so that <Benefit>

- Historias de usuario
- Product backlog
 - Sprint backlog



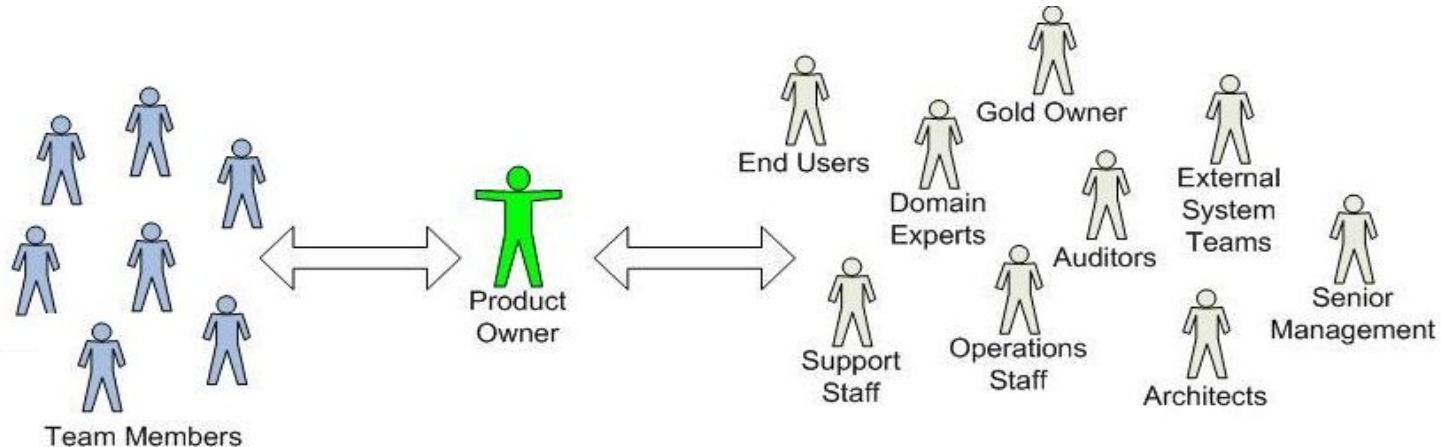
	Item #	Description	Est	By
Very High				
	1	Finish database versioning	16	KH
	2	Get rid of unneeded shared Java in database	8	KH
	-	Add licensing	-	-
	3	Concurrent user licensing	16	TG
	4	Demo / Eval licensing	16	TG
		Analysis Manager		
	5	File formats we support are out of date	160	TG
	6	Round-trip Analyses	250	MC
High				
	-	Enforce unique names	-	-
	7	In main application	24	KH
	8	In import	24	AM
	-	Admin Program	-	-
	9	Delete users	4	JM
	-	Analysis Manager	-	-
		When items are removed from an analysis, they should show up again in the pick list in lower 1/2 of the analysis tab	8	TG
	10	Query	-	-
	11	Support for wildcards when searching	16	T&A
	12	Sorting of number attributes to handle negative numbers	16	T&A
	13	Horizontal scrolling	12	T&A
	-	Population Genetics	-	-
	14	Frequency Manager	400	T&M
	15	Query Tool	400	T&M
	16	Additional Editors (which ones)	240	T&M
	17	Study Variable Manager	240	T&M
	18	Haplotypes	320	T&M
	19	Add icons for v1.1 or 2.0	-	-

Scrum: roles

- Product Owner
- Scrum Team
- Scrum Master
- Cliente
- Dirección (management)

Propietario del Producto (Product Owner)

- La “voz” del cliente para el equipo
 - Define los requisitos/historias de usuario
 - Prioriza los requisitos en función del valor que proporcionan al cliente
 - Ajusta las prioridades en cada sprint si es necesario
 - Acepta o deniega el resultado al final del sprint en el sprint review



Scrum Team

- Formados por 4-9 individuos
- “Cross-functional”: Programmers, testers, user experience designers, etc.
- Equipos self-organized



Scrum Master

SERVANT LEADER - El Scrum Master NO ES un Jefe de Proyecto “tradicional”.



- Elimina impedimentos
- Asegura que el equipo es cross-functional (tiene todo el conocimiento necesario para llevar a cabo el trabajo).
- Protege al equipo de interferencias externas
- Ayuda al equipo a mejorar (coach)
- Asegura que se siguen cada uno de los pasos del ciclo de vida Scrum acordado.

Gestión de Proyectos en Modelos Ágiles

Agile Teams



Servant Leader
Facilitator

No existe la figura de Jefe de proyecto tal cual

- El equipo planifica y estima el trabajo que se realiza en cada sprint.
- El equipo se auto gestiona para llevar a cabo el trabajo de acuerdo con los objetivos del sprint y el plan definido en el sprint planning.
- El equipo coordina/controla su propio trabajo ayudándose de técnicas como el meeting diario.
- El equipo es responsable de la documentación del proyecto.
- El Scrum master ayuda a eliminar impedimentos y motiva a los miembros del proyecto.
- Sin embargo, en proyectos grandes ciertas tareas de gestión son realizadas fuera del equipo.

“Self-Organized”

Fuente: Nerur, S., Mahapatra, R., & Mangalaraj, G. (2005). Challenges of migrating to agile methodologies. *Communications of the ACM*, 48(5), 72-78.

	Traditional	Agile
Fundamental Assumptions	Systems are fully specifiable, predictable, and can be built through meticulous and extensive planning.	High-quality, adaptive software can be developed by small teams using the principles of continuous design improvement and testing based on rapid feedback and change.
Control	Process centric	People centric
Management Style	Command-and-control	Leadership-and-collaboration
Knowledge Management	Explicit	Tacit
Role Assignment	Individual—favors specialization	Self-organizing teams—encourages role interchangeability
Communication	Formal	Informal
Customer's Role	Important	Critical
Project Cycle	Guided by tasks or activities	Guided by product features
Development Model	Life cycle model (Waterfall, Spiral, or some variation)	The evolutionary-delivery model
Desired Organizational Form/Structure	Mechanistic (bureaucratic with high formalization)	Organic (flexible and participative encouraging cooperative social action)
Technology	No restriction	Favors object-oriented technology

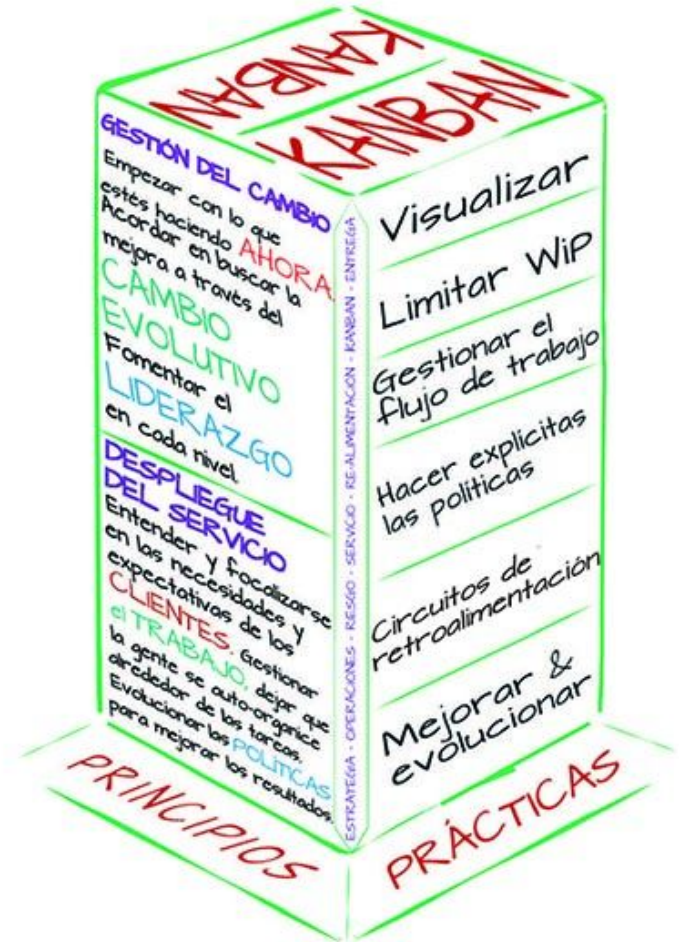
Kanban

Kanban - Orígenes

El método Kanban tiene sus orígenes en Japón, específicamente en la empresa Toyota, en la década de 1940. Originalmente, Kanban se utilizaba como un sistema de control de inventario para optimizar la producción en las fábricas. El término "Kanban" en japonés significa "tarjeta visual" o "señal", y se refiere a las tarjetas físicas que se utilizaban para indicar el flujo de trabajo y la disponibilidad de materiales en el proceso de fabricación.

En la década de 2000, el método Kanban fue adoptado en el ámbito del desarrollo de software por David J. Anderson y otros expertos en gestión ágil.

Principios y prácticas de Kanban



Kanban

La metodología Kanban se basa en una filosofía centrada en la mejora continua, donde las tareas se “extraen” de una lista de acciones pendientes en un flujo de trabajo constante.



Kanban - visualización del trabajo

Visualización del trabajo: La visualización es fundamental en Kanban. Se utiliza un tablero Kanban, generalmente dividido en columnas que representan diferentes etapas del flujo de trabajo, desde la idea hasta la entrega. Cada tarea o elemento de trabajo se representa como una tarjeta que se mueve a través del tablero a medida que avanza en el proceso.

Las columnas del tablero típicamente incluyen las etapas como "Por hacer", "En progreso", "En revisión" y "Completado".

Kanban - WIP

Limitación del trabajo en proceso (WIP): Se establece un límite en la cantidad de trabajo que se puede realizar simultáneamente en cada etapa del flujo de trabajo. Limitar el trabajo en proceso ayuda a evitar la sobrecarga de trabajo, reducir el tiempo de espera y mantener un flujo constante de trabajo.

Un desarrollador no puede empezar una nueva tarea hasta que haya completado o movido al menos una tarea al estado "Completado". Limitar el WIP ayuda a evitar la multitarea excesiva y a mantener un flujo de trabajo más fluido.

Kanban - Flujo continuo

Gestión del flujo: Kanban se centra en optimizar el flujo de trabajo, identificando cuellos de botella, eliminando desperdicios y maximizando la eficiencia en cada etapa del proceso. Se anima a los equipos a identificar y abordar los obstáculos que ralentizan el flujo de trabajo.

Ej. Durante una reunión regular de revisión del flujo, el equipo identifica que la etapa de revisión de código está siendo un cuello de botella. Deciden asignar más recursos para revisar el código o implementar un sistema de rotación de tareas para equilibrar la carga de trabajo

Kanban - Feedback

Retroalimentación continua y mejora: Kanban promueve la mejora continua mediante la revisión regular del proceso y el rendimiento del equipo. Se fomenta la retroalimentación continua de los clientes y los miembros del equipo para identificar áreas de mejora y realizar ajustes en el proceso.

Ej. Después de cada iteración de desarrollo, el equipo realiza una retrospectiva para analizar lo que salió bien, lo que salió mal y qué se puede mejorar. Basándose en esta retroalimentación, ajustan sus procesos y prácticas para la siguiente iteración, como mejorar la comunicación entre el equipo o implementar nuevas herramientas de prueba automatizadas.

Kanban - Gestión

Gestión explícita de políticas: Kanban requiere que las políticas y reglas del proceso de desarrollo de software sean claras y estén explícitamente definidas. Esto ayuda a garantizar que todos los miembros del equipo comprendan cómo se debe realizar el trabajo y cuáles son las expectativas en cada etapa del proceso.

Ej. establecer una política clara de revisión de código que indique que cada solicitud de extracción debe ser revisada por al menos dos miembros del equipo antes de fusionarse en la rama principal. Esta política garantiza la calidad del código y establece expectativas claras para todos los miembros del equipo.

Kanban - Mejora continua

Enfoque basado en la mejora evolutiva: Kanban adopta un enfoque pragmático y evolutivo para la implementación de cambios. En lugar de realizar cambios radicales, se fomenta la implementación de pequeñas mejoras incrementales de manera continua, lo que permite a los equipos adaptarse gradualmente y aprender de la experiencia

Ej. En lugar de cambiar todo el proceso de desarrollo de un proyecto de una sola vez, el equipo decide implementar una pequeña mejora cada semana. Por ejemplo, podrían comenzar por automatizar las pruebas de integración antes de pasar a la mejora del proceso de despliegue continuo en las siguientes semanas.

Kanban - Tablero

Kanban board:

- **Visualiza el flujo** de trabajo.
- **Limita el trabajo en progreso.**
- **Mide y optimiza.**

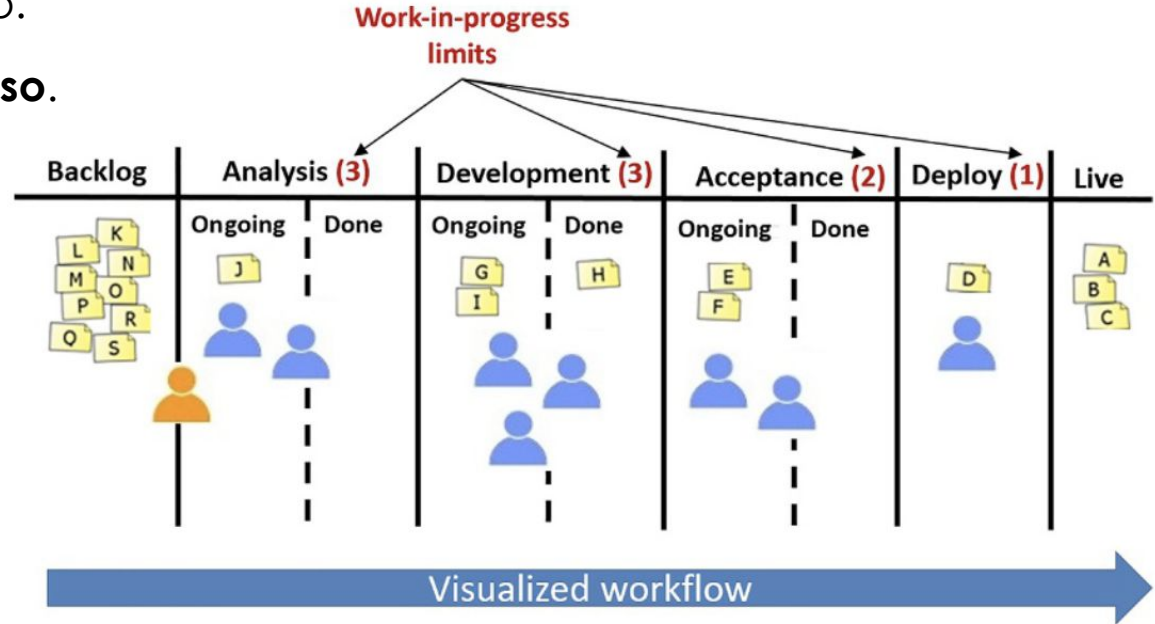
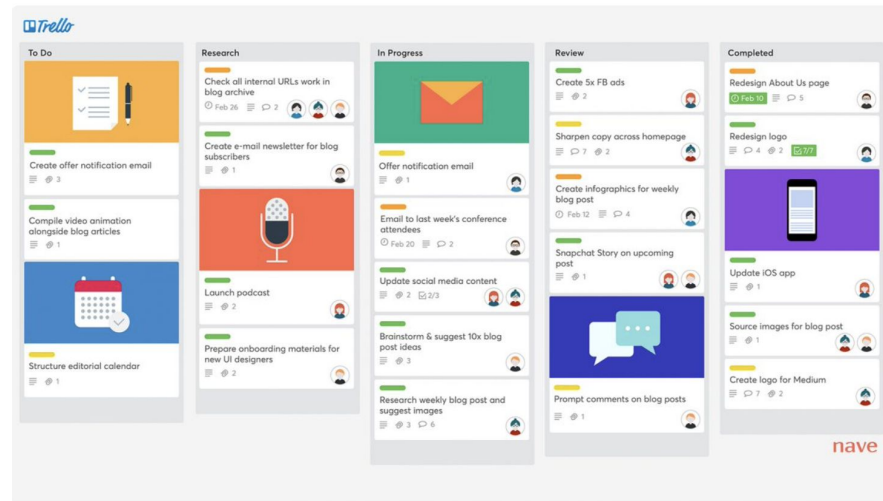
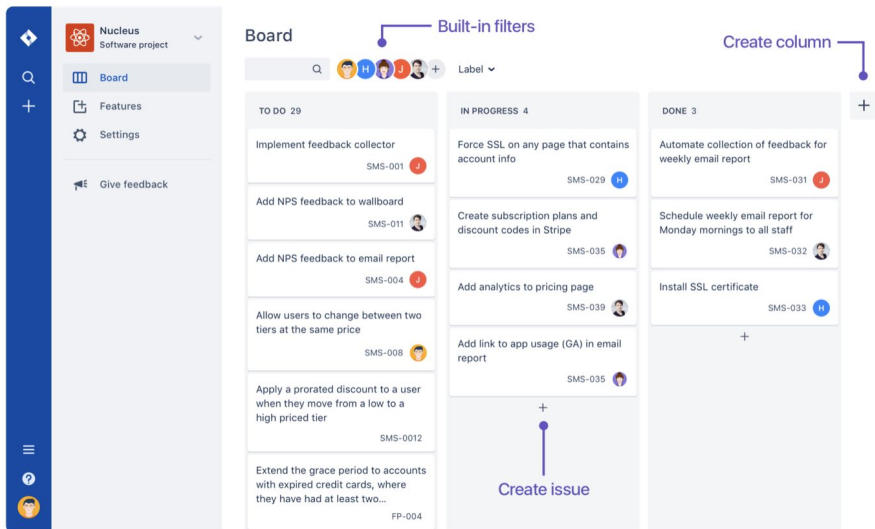


Fig. 10 Example of a Kanban board for software development.

Kanban - herramientas (no exclusivas para esa metodología)



<https://www.atlassian.com/es/software/jira>

<https://trello.com>

¿Cuándo implementar Kanban?

- Flujo de trabajo no lineal: Cuando el equipo enfrenta un flujo de trabajo impredecible o no lineal, donde las tareas surgen de manera irregular y no se ajustan bien a los ciclos de desarrollo predefinidos. Kanban es especialmente útil para proyectos de mantenimiento o soporte, donde las solicitudes de trabajo pueden surgir en cualquier momento y necesitan ser abordadas de manera rápida y eficiente.
- Necesidad de visibilidad y transparencia: Cuando se requiere una mayor visibilidad y transparencia en el proceso de desarrollo para que todos los miembros del equipo comprendan el estado de cada tarea y dónde se encuentra en el flujo de trabajo.
- Enfoque en mejora continua: Cuando el equipo busca un enfoque de mejora continua en su proceso de desarrollo, Kanban proporciona un marco para identificar y abordar los cuellos de botella, eliminar desperdicios y optimizar el flujo de trabajo de manera constante.

Lean Startup

Lean Startup - Orígenes

El enfoque Lean Startup se basa en una combinación de metodologías ágiles de desarrollo de software, principios de gestión de la calidad y conceptos de gestión lean, como los desarrollados por Toyota en su Sistema de Producción Toyota (TPS).

El término "Lean Startup" fue popularizado por Eric Ries a través de su libro "The Lean Startup", publicado en 2011. En este libro, Ries presenta los principios y prácticas del Lean Startup, que propone un enfoque para desarrollar productos y empresas de manera más eficiente y efectiva, minimizando el desperdicio y maximizando el aprendizaje validado.

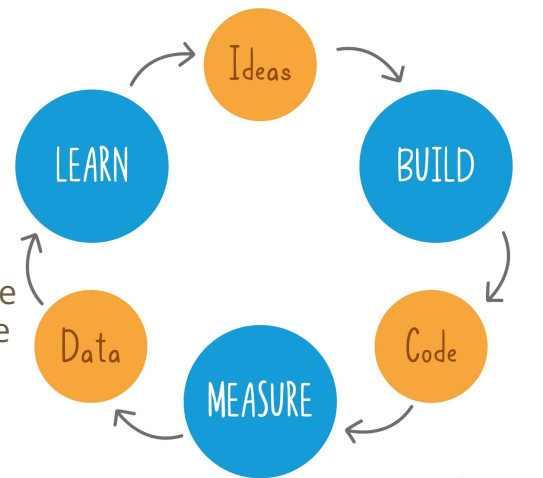


Lean Startup - Fundamentos

Validación de ideas mediante experimentación: En lugar de basarse únicamente en suposiciones y planificaciones extensas, Lean Startup promueve la validación de ideas a través de experimentos rápidos y económicos. Esto implica crear versiones mínimas de productos (MVP, por sus siglas en inglés) y someterlas a prueba en el mercado para recopilar datos reales sobre la respuesta de los clientes.

Iteración continua: La metodología Lean Startup aboga por un enfoque iterativo en el desarrollo de productos, donde se realizan ajustes y mejoras constantes en función de la retroalimentación del cliente y los datos recopilados durante los experimentos. Este enfoque permite adaptarse rápidamente a las necesidades y preferencias del mercado, evitando la sobreinversión en características o funcionalidades innecesarias.

Aprendizaje validado: Lean Startup se basa en la idea de que el aprendizaje es el objetivo principal del proceso de desarrollo de productos. Se enfoca en obtener y validar aprendizajes significativos sobre el mercado, los clientes y el producto a través de la experimentación y la recopilación de datos. Este enfoque permite a las empresas tomar decisiones más informadas y reducir el riesgo de fracaso al tiempo que maximiza las oportunidades de éxito.



Lean Startup - Actividades

Desarrollar una hipótesis: El proceso comienza con el desarrollo de una hipótesis sobre el problema que estás tratando de resolver, la solución propuesta y las suposiciones subyacentes. Esta hipótesis actúa como una guía inicial para tus esfuerzos y experimentos.

Ejemplo: Las personas están interesadas en comprar zapatos por canales online

Lean Startup - Actividades

Construir un MVP (Producto Mínimo Viable): Una vez que tengas una hipótesis clara, construye un MVP, que es la versión más simple de tu producto o servicio que te permitirá probar tu hipótesis con los clientes reales. El MVP debe ser lo suficientemente básico como para ser construido y lanzado rápidamente, pero lo suficientemente completo como para proporcionar una experiencia de usuario significativa.

Ejemplo: Una página web con catálogo de zapatos y gestionar la compra. El catálogo se basa en una planilla y los pedidos también. Una persona revisa el listado y compra en tiendas físicas.

Lean Startup - Actividades

Lanzar el MVP y recopilar datos: Lanza tu MVP al mercado y comienza a recopilar datos sobre cómo los clientes interactúan con él. Utiliza métricas clave para evaluar el rendimiento del MVP y para determinar si tu hipótesis es válida o no. Estas métricas pueden incluir tasas de conversión, tiempos de uso, retroalimentación del cliente, etc.

Ejemplo: Campaña de marketing para promocionar la página en un sector en particular.

Lean Startup - Actividades

Analizar y aprender: Basándote en los datos recopilados, analiza los resultados y extrae lecciones aprendidas. ¿Tu hipótesis fue validada? ¿Qué funcionó bien y qué no? ¿Qué puedes aprender de esta experiencia para mejorar tu producto o estrategia? Este análisis te ayudará a tomar decisiones informadas sobre cómo proceder.

Ejemplo: ¿Cuántas personas accedieron a la página? ¿Cuántas compraron? ¿Cuántas compraron más de una vez?

- **Porcentaje de activación:** Medimos la cantidad de clientes que han pasado de estar interesados en nuestro producto a ser clientes potenciales (no consumidores todavía). Será necesario conocer sus motivaciones y el tiempo que ha pasado desde que nos conocieron hasta que comenzaron a plantearse la opción de compra.
- **Engagement:** Se trata de conocer la **cantidad** de usos que se le da a nuestro producto. Es decir, si el cliente medio puede prescindir de nosotros durante un tiempo, o si somos imprescindibles en su día a día.
- **CHURN:** Mide la rotación de clientes. La diferencia entre los clientes que nos han abandonado y aquellos que han adquirido nuestro producto, todo esto marcando un plazo determinado.
- **Conversión:** Esta es una métrica clave en cualquier negocio. Mide las personas que pasaron de ser clientes potenciales a ser consumidores. Es muy importante conocer el por qué de la decisión.
- **Coste por cliente:** Es la parte de la inversión que nos ha costado atraer a un cliente. Se mide por períodos y se utiliza para comparar diferentes fases del producto.
- **Cash Burn Rate:** Es la suma del dinero que invertimos, o perdemos, mensualmente en sueldos, desarrollo y mejora, publicidad, ... Nos ayuda a saber en qué y cómo gastamos nuestro dinero.
- **Referencia:** Con este KPI medimos los clientes que han venido recomendados por una persona que ya nos conocía.

Lean Startup - Actividades

Pivotar o perseverar: Basándote en tus aprendizajes, decide si debes pivotar o perseverar. Un pivotaje implica hacer un cambio significativo en tu estrategia o producto en respuesta a los aprendizajes obtenidos. Por otro lado, la perseverancia implica continuar con la misma estrategia o producto, pero con ajustes basados en los aprendizajes obtenidos.

Ejemplo: Netflix empezó enviando películas y material audiovisual por DVD y luego optó por digitalizar esos mismos productos y ofrecerlos mediante el servicio de streaming.

Lean Startup - Actividades

Iterar y repetir: Basándote en tus decisiones, itera en el proceso. Esto significa volver al ciclo, desarrollar una nueva hipótesis basada en tus aprendizajes anteriores, construir un nuevo MVP, lanzarlo al mercado, recopilar datos y continuar aprendiendo y mejorando.

¿Cuándo implementar Lean Startup?

Los tres casos clave en los que típicamente es apropiado considerar el uso de Lean Startup:

- Desarrollo de un nuevo producto o servicio: Cuando estás creando algo completamente nuevo y desconocido, donde la incertidumbre es alta y necesitas validar rápidamente tus ideas en el mercado.
- Entrar en un mercado competitivo o desconocido: Cuando estás entrando en un mercado nuevo o altamente competitivo y necesitas comprender rápidamente las necesidades y preferencias de los clientes para ajustar tu enfoque.
- Startups o empresas emergentes con recursos limitados: Cuando estás operando con recursos limitados y necesitas maximizar la eficiencia en el uso de esos recursos, centrándote en lo que realmente importa para los clientes y eliminando actividades que no agregan valor.

¿Cuándo utilizar “Tradicional” y cuándo utilizar Ágil?

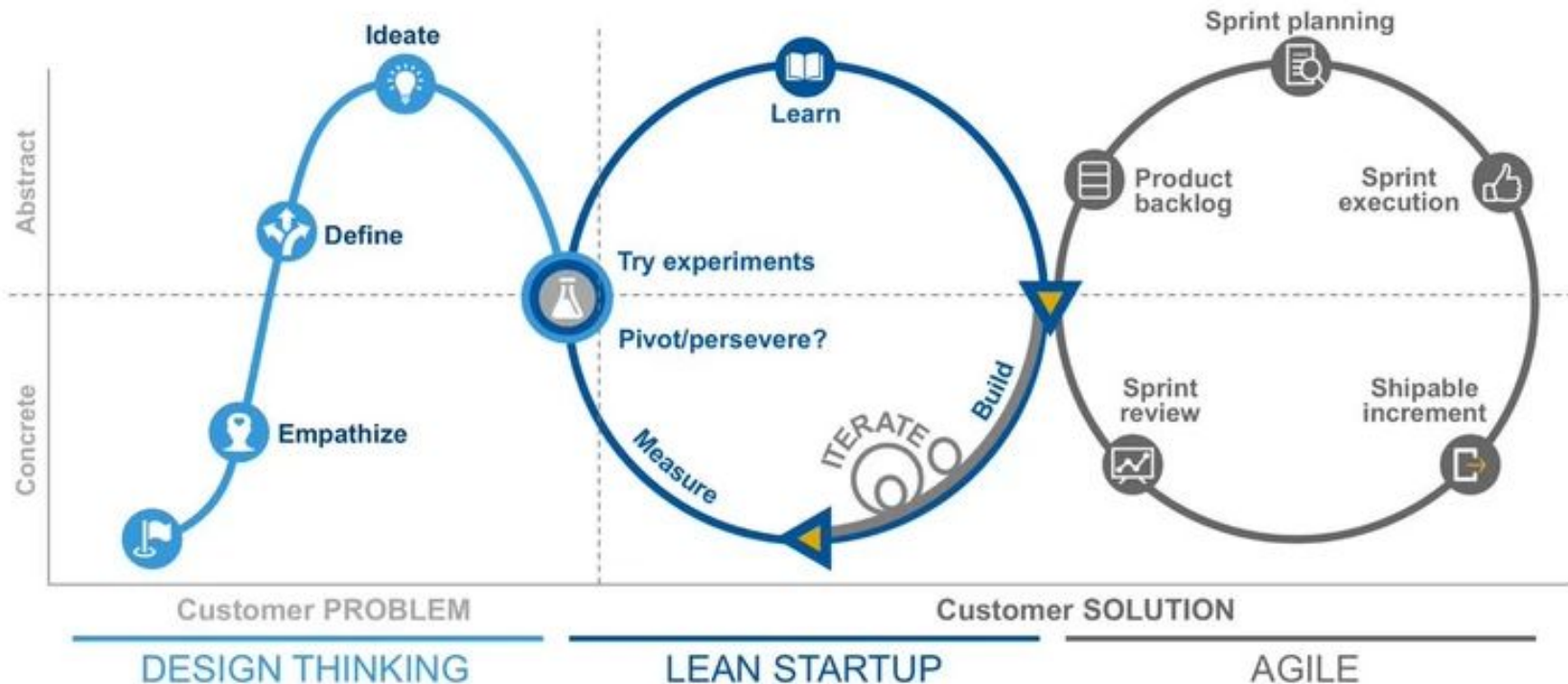
- **Ciclos de Vida Tradicionales:**

- Proyectos con un problema conocido y una solución al mismo bien definida.
- Podemos establecer los requisitos al comienzo del proyecto.
- Proyectos en los que se pueden predecir costes/necesidades.
- Proyectos con contratos estrictos (alcance, tiempo y presupuesto fijos).
- Software crítico.

- **Desarrollo Ágil:**

- Entornos cambiantes.
- Proyectos en los que hay mucha incertidumbre.
- Proyectos en los que el Time-to-market es importante (innovación).
- Proyectos en los que el cliente/usuario pueda estar implicado.

Combine Design Thinking, Lean Startup and Agile



Para complementar

Capítulos 3 y 4 de Somerville

Quiz en la siguiente clase con las siguientes preguntas

¿Cuáles prácticas ágiles ya conocía?

¿Cuáles metodologías ágiles ya conocía?

¿Cuáles prácticas o metodologías más le gusta y aplicaría en sus actuales o siguientes proyectos? ¿Porqué?

¿Cuáles prácticas o metodologías menos le gusta o con más dificultad la incorporaría en sus proyectos? ¿Porqué?